

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 20%

QUESTIONS: 15

Total Marks: 25

Annexure A Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I V.Madhulika(192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student

Q1. Pseudocode to read a dataset from a file and preprocess it (handle missing values, remove duplicates, normalize numerical columns, standardize categorical values, indexing, filtering, save output).

Answer:

INPUT: file_path

1. $df \leftarrow \text{READ_FILE}(\text{file_path})$
2. FOR each column IN df:
 IF column is numeric AND has missing values THEN FILL with $\text{MEDIAN}(\text{column})$
 ELSE IF column is categorical AND has missing values THEN FILL with $\text{MODE}(\text{column})$
3. $df \leftarrow \text{REMOVE_DUPLICATES}(df, \text{subset} = \text{primary_key})$
4. FOR each numeric_column IN df:
 $df[\text{numeric_column}] \leftarrow (df[\text{numeric_column}] - \text{MIN}) / (\text{MAX} - \text{MIN})$ // normalize
5. FOR each categorical_column IN df:
 $df[\text{categorical_column}] \leftarrow \text{STRIP_LOWER}(df[\text{categorical_column}])$
 $df[\text{categorical_column}] \leftarrow \text{MAP_TO_STANDARD_LABELS}(df[\text{categorical_column}])$
6. $\text{valid_mask} \leftarrow \text{APPLY_RANGE/FORMAT_CHECKS}(df)$
7. $df \leftarrow df[\text{valid_mask}]$ // filter invalid entries
8. $df \leftarrow \text{RESET_INDEX}(df)$ // consistent indexing
9. $\text{SAVE}(df, \text{output_path})$

OUTPUT: cleaned dataset file

Efficiency note: perform steps 2, 4, 5 as vectorized column operations, not row-wise loops.

Q2. Pseudocode to compute mean, median, standard deviation, and variance efficiently, with filtering and handling of edge cases (empty arrays, extreme outliers).

Answer:

INPUT: array A

1. IF $\text{LENGTH}(A) == 0$ THEN RETURN error 'empty array'
2. $A \leftarrow \text{REMOVE_NULLS}(A)$
3. $Q1, Q3 \leftarrow \text{PERCENTILE}(A, 25), \text{PERCENTILE}(A, 75)$
4. $IQR \leftarrow Q3 - Q1$
5. $\text{filtered} \leftarrow [x \text{ FOR } x \text{ IN } A \text{ IF } (Q1 - 1.5 \times IQR) \leq x \leq (Q3 + 1.5 \times IQR)]$ // exclude extreme outliers
6. IF $\text{LENGTH}(\text{filtered}) == 0$ THEN RETURN error 'all values filtered as outliers'
7. $\text{mean} \leftarrow \text{SUM}(\text{filtered}) / \text{LENGTH}(\text{filtered})$
8. $\text{sorted_arr} \leftarrow \text{SORT}(\text{filtered})$
9. $\text{median} \leftarrow \text{MIDDLE_VALUE}(\text{sorted_arr})$
10. $\text{variance} \leftarrow \text{SUM}((x - \text{mean})^2 \text{ FOR } x \text{ IN } \text{filtered}) / (\text{LENGTH}(\text{filtered}) - 1)$
11. $\text{std_dev} \leftarrow \text{SQRT}(\text{variance})$
12. RETURN mean, median, std_dev, variance

Efficiency note: compute SUM and SUM_OF_SQUARES in one pass to derive mean and variance together, avoiding two separate traversals.

Q3. Pseudocode to combine two or more DataFrames: align indices, merge on a common key, remove duplicates, validate integrity, with grouping and sorting.

Answer:

INPUT: DataFrames D1, D2, key

1. $D1 \leftarrow \text{RESET_INDEX}(D1)$; $D2 \leftarrow \text{RESET_INDEX}(D2)$ // align indices
2. VERIFY key EXISTS IN D1 AND D2
3. $\text{merged} \leftarrow \text{OUTER_JOIN}(D1, D2, \text{on} = \text{key})$
4. $\text{merged} \leftarrow \text{REMOVE_DUPLICATES}(\text{merged}, \text{subset} = \text{key})$
5. $\text{unmatched} \leftarrow \text{merged}[D1_columns \text{ ARE NULL OR } D2_columns \text{ ARE NULL}]$
6. LOG unmatched RECORDS for review
7. $\text{merged} \leftarrow \text{GROUP_BY}(\text{merged}, \text{key}).\text{AGGREGATE}(\dots)$ // consolidate related rows
8. $\text{merged} \leftarrow \text{SORT}(\text{merged}, \text{by} = \text{key})$
9. $\text{ASSERT COUNT}(\text{merged}) \leq \text{COUNT}(D1) + \text{COUNT}(D2)$ // integrity check, no unexpected row explosion
10. RETURN merged

OUTPUT: single validated, sorted, de-duplicated DataFrame

Q4. Pseudocode to filter a dataset by multiple conditions (value ranges, categories) and sort the results, optimized for large datasets.

Answer:

INPUT: DataFrame df, conditions $C = \{\text{range_conditions}, \text{category_conditions}\}$, sort_column

1. mask $\leftarrow$ TRUE for all rows
2. FOR each condition c IN C:
 mask $\leftarrow$ mask AND EVALUATE(c, df) // combine with logical AND, vectorized
3. $\text{filtered} \leftarrow \text{df}[\text{mask}]$ // boolean-index in one pass, not row loop
4. $\text{sorted_df} \leftarrow \text{SORT}(\text{filtered}, \text{by} = \text{sort_column}, \text{ascending} = \text{TRUE})$
5. RETURN sorted_df

OUTPUT: filtered and sorted dataset

Optimization note: evaluate cheapest/most-selective conditions first to shrink the dataset early; use indexed columns for the sort where available to avoid a full re-sort.

Q5. Pseudocode for a complete data analysis workflow: read files, convert to DataFrames, group/aggregate, generate plots, handle I/O errors.

Answer:

INPUT: list of file_paths

1. all_data $\leftarrow$ EMPTY LIST
2. FOR each path IN file_paths:
 TRY:
 df $\leftarrow$ READ_FILE(path)
 APPEND df TO all_data
 CATCH error:
 LOG 'failed to read ' + path; CONTINUE
3. combined $\leftarrow$ CONCAT(all_data)

4. combined $\leftarrow$ STANDARDIZE_COLUMNS(combined)
5. combined $\leftarrow$ APPLY_TRANSFORMATIONS(combined) // cleaning, type conversion
6. summary $\leftarrow$ GROUP_BY(combined, key_column).AGGREGATE(metrics)
7. summary $\leftarrow$ SORT(summary, by = metric)
8. PLOT(summary) // bar/line chart based on data type
9. SAVE_PLOT(output_path)

OUTPUT: summarized DataFrame and visualization file

Q6. Pseudocode for cleaning/transforming time-series data: handle missing timestamps, interpolate, remove outliers, resample at fixed intervals.

Answer:

INPUT: time-series DataFrame df with 'timestamp' and 'value'

1. df $\leftarrow$ SET_INDEX(df, 'timestamp')
2. df $\leftarrow$ SORT_BY_INDEX(df) // chronological consistency
3. full_index $\leftarrow$ DATE_RANGE(MIN(timestamp), MAX(timestamp), freq = target_interval)
4. df $\leftarrow$ REINDEX(df, full_index) // exposes missing timestamps as NaN
5. df['value'] $\leftarrow$ INTERPOLATE(df['value'], method='time')
6. Q1, Q3 $\leftarrow$ PERCENTILE(df['value'], 25), PERCENTILE(df['value'], 75)
7. df $\leftarrow$ df[(df['value'] $\geq$ Q1 - 1.5×IQR) AND (df['value'] $\leq$ Q3 + 1.5×IQR)] // remove outliers
8. df $\leftarrow$ RESAMPLE(df, freq = target_interval).MEAN()
9. RETURN df

OUTPUT: cleaned, evenly-spaced time-series

Q7. Pseudocode to implement feature selection: compute correlation, remove redundant/irrelevant attributes, retain significant features (with normalization and threshold-based filtering).

Answer:

INPUT: DataFrame df, target column y, threshold t

1. df $\leftarrow$ NORMALIZE(df) // Z-score each numeric feature
2. corr_matrix $\leftarrow$ CORRELATION(df)
3. redundant $\leftarrow$ EMPTY SET
4. FOR each pair (fi, fj) IN corr_matrix WHERE fi $\neq$ fj:
IF ABS(corr_matrix[fi][fj]) > t THEN ADD fj TO redundant
5. df $\leftarrow$ DROP_COLUMNS(df, redundant)
6. target_corr $\leftarrow$ CORRELATION(df, y)
7. selected $\leftarrow$ [f FOR f IN df.columns IF ABS(target_corr[f]) > t_relevance]
8. df_final $\leftarrow$ df[selected]
9. VALIDATE df_final has no NULLs and consistent shape
10. RETURN df_final

OUTPUT: reduced feature set for modeling

Q8. Pseudocode for data partitioning into training, validation, testing sets with randomization, stratification, and no data leakage.

Answer:

INPUT: DataFrame df, target y, split ratios (train, val, test), stratify_flag

1. $df \leftarrow \text{SHUFFLE}(df, \text{random_seed} = \text{fixed_seed})$
2. IF stratify_flag THEN
 groups $\leftarrow \text{GROUP_BY}(df, y)$
 FOR each group IN groups:
 SPLIT group INTO train_g, val_g, test_g USING ratios
 APPEND to train, val, test respectively
 ELSE
 SPLIT df INTO train, val, test USING ratios (sequential after shuffle)
3. ASSERT INTERSECTION(train.index, val.index, test.index) == EMPTY // no leakage
4. VERIFY class distribution in train/val/test approximately matches original (if stratified)
5. RETURN train, val, test

OUTPUT: three non-overlapping, class-balanced partitions

Q9. Pseudocode to detect and handle outliers using Z-score or IQR, deciding whether to remove or cap them.

Answer:

INPUT: array A, method ('zscore' or 'iqr'), action ('remove' or 'cap')

1. IF method == 'zscore':
 mean $\leftarrow \text{MEAN}(A)$; std $\leftarrow \text{STD}(A)$
 scores $\leftarrow (A - \text{mean}) / \text{std}$
 outlier_mask $\leftarrow \text{ABS}(\text{scores}) > 3$
2. ELSE IF method == 'iqr':
 Q1, Q3 $\leftarrow \text{PERCENTILE}(A, 25), \text{PERCENTILE}(A, 75)$; IQR $\leftarrow Q3 - Q1$
 lower $\leftarrow Q1 - 1.5 \times \text{IQR}$; upper $\leftarrow Q3 + 1.5 \times \text{IQR}$
 outlier_mask $\leftarrow (A < \text{lower}) \text{ OR } (A > \text{upper})$
3. IF action == 'remove':
 A $\leftarrow A[\text{NOT outlier_mask}]$
 ELSE IF action == 'cap':
 A $\leftarrow \text{CLIP}(A, \text{lower}, \text{upper})$
4. VALIDATE resulting distribution (no NaN, consistent length or logged removal count)
5. RETURN A

OUTPUT: outlier-handled, reliable dataset

Q10. Pseudocode for data aggregation and summarization across multiple categories (group, compute sum/average/count, sort, validate).

Answer:

INPUT: DataFrame df, group_column, value_column

1. groups $\leftarrow \text{GROUP_BY}(df, \text{group_column})$

```

2. summary ← EMPTY TABLE
3. FOR each group IN groups:
    total ← SUM(group[value_column])
    avg ← AVERAGE(group[value_column])
    cnt ← COUNT(group[value_column])
    APPEND (group_key, total, avg, cnt) TO summary
4. summary ← SORT(summary, by = total, descending = TRUE)
5. ASSERT SUM(summary.cnt) == COUNT(df) // validate no rows lost
6. RETURN summary
OUTPUT: sorted, validated summary report

```

Q11. Pseudocode for data normalization/scaling supporting Min-Max scaling and Z-score normalization, with column identification and validation.

Answer:

```

INPUT: DataFrame df, method ('minmax' or 'zscore')
1. numeric_cols ← IDENTIFY_NUMERIC_COLUMNS(df)
2. FOR each col IN numeric_cols:
    IF col HAS missing values THEN FILL with MEDIAN(col)
3. FOR each col IN numeric_cols:
    IF method == 'minmax':
        df[col] ← (df[col] - MIN(col)) / (MAX(col) - MIN(col))
    ELSE IF method == 'zscore':
        df[col] ← (df[col] - MEAN(col)) / STD(col)
4. FOR each col IN numeric_cols:
    IF method == 'minmax' THEN ASSERT ALL values IN [0,1]
    ELSE ASSERT MEAN(df[col]) ≈ 0 AND STD(df[col]) ≈ 1
5. RETURN df
OUTPUT: scaled dataset with all numeric features validated in range

```

Q12. Pseudocode for dimensionality reduction using PCA: standardize, compute covariance matrix, extract eigenvalues/eigenvectors, select principal components.

Answer:

```

INPUT: DataFrame df, num_components k
1. df_std ← (df - MEAN(df)) / STD(df) // standardize
2. cov_matrix ← COVARIANCE(df_std)
3. eigenvalues, eigenvectors ← EIGEN_DECOMPOSITION(cov_matrix)
4. SORT eigenvalues (and corresponding eigenvectors) DESCENDING
5. explained_variance ← eigenvalues / SUM(eigenvalues)
6. top_k_vectors ← eigenvectors[:, 0:k]
7. reduced ← MATRIX_MULTIPLY(df_std, top_k_vectors)
8. VALIDATE SHAPE(reduced) == (ROWS(df), k)
9. VALIDATE SUM(explained_variance[0:k]) ≥ desired_threshold (e.g., 0.95)

```

10. RETURN reduced

OUTPUT: reduced-dimension dataset preserving maximum variance

Q13. Pseudocode for data merging and conflict resolution from multiple sources: detect conflicts, apply resolution strategy (priority/averaging), ensure consistency.

Answer:

INPUT: DataFrames D1 (priority 1), D2 (priority 2), key

1. merged $\leftarrow$ OUTER_JOIN(D1, D2, on = key, suffixes = ('_1', '_2'))
2. FOR each shared_column c IN D1 AND D2:
 conflict_mask $\leftarrow$ (merged[c+'_1'] $\neq$ merged[c+'_2']) AND BOTH NOT NULL
 FLAG rows WHERE conflict_mask
3. FOR each shared_column c:
 IF resolution_strategy == 'priority':
 merged[c] $\leftarrow$ COALESCE(merged[c+'_1'], merged[c+'_2']) // prefer D1
 ELSE IF resolution_strategy == 'averaging' AND c IS NUMERIC:
 merged[c] $\leftarrow$ AVERAGE(merged[c+'_1'], merged[c+'_2'])
4. DROP intermediate suffixed columns
5. VALIDATE no key duplicates AND no unresolved NULLS in required fields
6. RETURN merged

OUTPUT: single consistent dataset with conflicts resolved and logged

Q14. Pseudocode for rolling window computations (moving average, rolling variance) on time-series data, handling boundaries and missing values, optimized for large datasets.

Answer:

INPUT: array A (time-ordered), window_size w

1. A $\leftarrow$ FILL_OR_FLAG_MISSING(A)
2. n $\leftarrow$ LENGTH(A)
3. moving_avg $\leftarrow$ ARRAY of size n, initialized NULL
4. running_sum $\leftarrow$ SUM(A[0 : w])
5. moving_avg[w-1] $\leftarrow$ running_sum / w
6. FOR i FROM w TO n-1:
 running_sum $\leftarrow$ running_sum - A[i-w] + A[i] // incremental update, O(1) per step
 moving_avg[i] $\leftarrow$ running_sum / w
7. rolling_var $\leftarrow$ ARRAY of size n, initialized NULL
8. FOR i FROM w-1 TO n-1:
 window $\leftarrow$ A[i-w+1 : i+1]
 rolling_var[i] $\leftarrow$ VARIANCE(window)
9. FOR i FROM 0 TO w-2: moving_avg[i], rolling_var[i] $\leftarrow$ NULL // boundary handling
10. RETURN moving_avg, rolling_var

OUTPUT: aligned moving-average and rolling-variance series

Optimization note: incremental sum update (step 6) avoids re-summing the full window each step, giving O(n) instead of O(n·w).

Q15. Pseudocode for data validation and integrity checks before analysis: verify types, detect missing/invalid entries, enforce constraints, generate a validation report.

Answer:

INPUT: DataFrame df, schema (expected types, valid ranges per column)

```

1. report ← EMPTY LIST
2. FOR each column IN df:
    IF ACTUAL_TYPE(column) ≠ schema[column].expected_type THEN
        LOG (column, 'type mismatch') TO report
3. FOR each column IN df:
    missing_count ← COUNT(NULLS IN column)
    IF missing_count > 0 THEN LOG (column, missing_count, 'missing values') TO report
4. FOR each column WITH defined range IN schema:
    invalid_rows ← df[ (df[column] < schema[column].min) OR (df[column] >
schema[column].max) ]
    IF COUNT(invalid_rows) > 0 THEN LOG (column, COUNT(invalid_rows), 'out of range')
TO report
5. valid_mask ← COMBINE all pass/fail conditions across rows
6. clean_df ← df[valid_mask]
7. WRITE report TO validation_report.log
8. RETURN clean_df, report
OUTPUT: validated dataset plus a detailed validation/error report

```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand